

Image Generation with a Diffusion Model

What worked, what didn't, and what the samples actually looked like

Deep Learning, Assignment 5 (Bonus)

Talal, MSDS25011

1. What I Set Out to Do

The goal was to train a diffusion model (DDPM) from scratch and have it generate images of five animal classes cat, dog, elephant, lion, and panda. The catch, which ended up mattering more than anything else, is how little data there was to work with: only **644 images in total**, roughly 129 per class. That is a tiny amount for a generative model, and most of what follows is really the story of pushing a small dataset as far as it will go.

The model predicts the noise added at each of 1000 diffusion steps and is trained with a simple L2 loss between the predicted and the real noise. To actually generate an image, I start from pure random noise and run the reverse process all the way back to a clean picture.

2. The Model Under the Hood

The generator is a **U-Net** that takes a noisy image plus the current timestep and predicts the noise that was added. It is deliberately compact, about **3.9 million parameters**, which suits both the small dataset and a from-scratch training budget.

2.1 Timestep conditioning

The model has to behave differently depending on how much noise is present, so the timestep is fed in everywhere. Each step is turned into a **sinusoidal embedding** (the same idea as positional encodings in transformers), passed through a small MLP, and then injected into every convolutional block. This is what lets one shared network handle all 1000 noise levels.

2.2 Encoder-decoder with skip connections

The encoder downsamples the image through three stages, with the channel width growing **64 → 128 → 256** as the spatial resolution shrinks. The decoder mirrors that, upsampling back to full resolution. Crucially, **skip connections** carry fine detail from each encoder stage straight across to the matching decoder stage, so the model doesn't lose sharp, high-frequency information while it reasons about the coarse layout in the middle. Each block is a double convolution with **GroupNorm** and **SiLU** activations, which keep training stable at this small batch size.

2.3 Self-attention at the bottleneck

At the lowest-resolution stage, the bottleneck, I added a **self-attention** block. Convolutions only see a local neighbourhood, so attention is what gives the network a global view: it can relate one part of the image to another and keep the overall composition coherent (a body in the middle, background around it). This is a big part of why the epoch-300 samples hold together as scenes rather than dissolving into texture.

2.4 Diffusion process

Training uses the standard DDPM setup with **1000 timesteps** and a **cosine noise schedule**. The forward process adds noise to a clean image in closed form (so any timestep can be sampled directly, without iterating), and the model is trained to predict that noise under an L2 loss. Generation runs the learned reverse process step by step from pure noise back to an image, and sampling is done from the **EMA-averaged** weights rather than the raw ones for cleaner output.

3. Reading the Loss Curve

The first thing that stood out was how quickly the loss dropped. Within the first ten epochs or so it fell from about 0.43 to under 0.05, and from then on it basically flat-lined around **0.025 to 0.03** for the rest of training. At first this looked like good news, but it is a bit of a trap: in diffusion training the average loss gets low fast because most randomly-sampled timesteps are heavy-noise steps where guessing the noise is easy. A low number here does not mean the pictures look good yet.

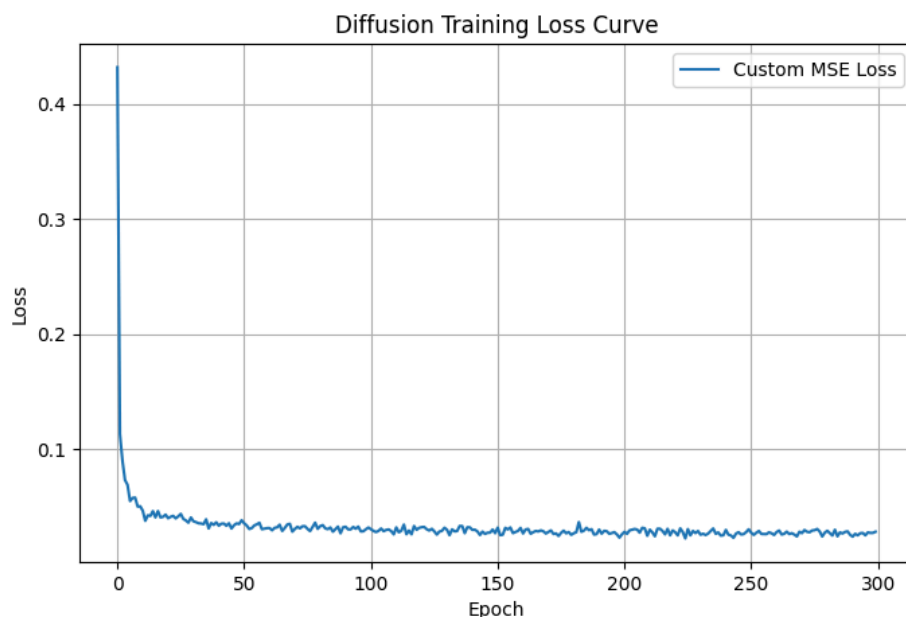


Figure 1. Training loss over epochs, a steep early drop, then a long flat plateau.

The more useful takeaway is that a **flat loss curve did not mean the model had stopped improving**. The actual image quality kept getting better for hundreds of epochs after the loss had already levelled off. All

those later gains come from the harder, low-noise steps that barely move the average. The flatness also tells me training was stable, no spikes, no divergence, once I had gradient clipping and a clean learning-rate decay in place.

4. How the Samples Evolved

This is the part I found most interesting. Early on the model can only produce coloured blobs; by the end it produces something that reads as an animal in a natural scene, and, again, the loss number hardly changed between those two points.

4.1 Epoch 50, just colour and texture

At fifty epochs the samples are soft clouds of green and brown. The model has clearly picked up the **colour palette and the texture** of the dataset, leafy greens, earthy browns, animal-fur greys, but there is no actual shape, nothing you could point at and call an animal, and no separation between subject and background. It has learned what the data feels like locally without any sense of overall layout.

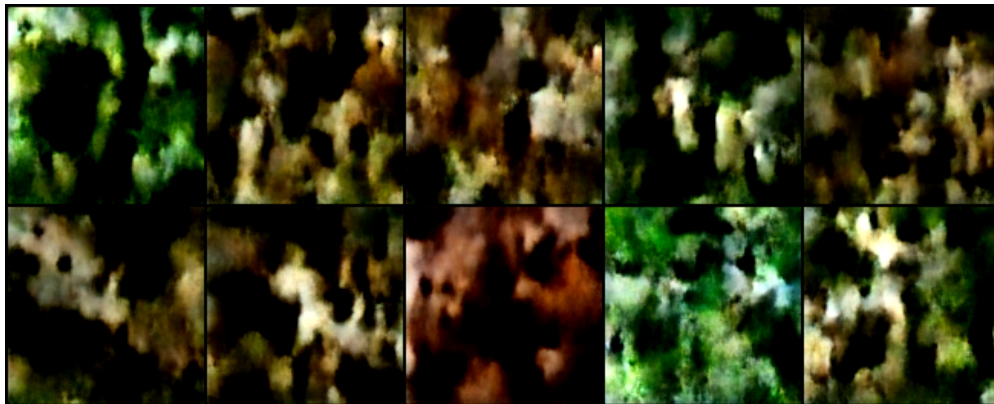


Figure 2. Epoch 50, right colours, right texture, no recognisable structure.

4.2 Epoch 300, shapes start to appear

By three hundred epochs the same model is producing much more organised images. You can see **subjects standing out against backgrounds**, animal-shaped forms against green foliage. There are black-and-white patches that look distinctly panda-like and grey masses that read as elephant-ish. Nothing is razor sharp, and I wouldn't claim every sample is unambiguously one species, but the jump from Figure 2 to Figure 3 is real and obvious, and it happened with almost no movement in the loss. That contrast is the main result I took away from this assignment.



Figure 3. Epoch 300, coherent scenes with animal-like forms and clear figure/background separation.

5. What Changed When I Changed Things

Along the way I varied a number of settings. Here is what each one actually did.

5.1 Learning rate

I started with a fairly aggressive learning rate of **1e-3**. It trained fast but the samples were unstable and washed-out, flat fields of colour with little structure. Dropping it to **2e-4**, which is the value people typically use for DDPMs, gave much steadier training and better-formed samples. Notably, the high rate made the loss number fall quickly while the images got worse, which is the same lesson as before: the loss and the visual quality are only loosely connected here.

5.2 Learning-rate schedule

I also tried a cosine schedule with warm restarts. That turned out to be a mistake, every restart kicked the learning rate back up and visibly disturbed training each time. Switching to a plain **cosine decay** across the whole run smoothed everything out and gave the clean curve in Figure 1.

5.3 Amount of data

Early on I was accidentally training on a capped subset of only about **100 images**, and the model produced nothing but noise, it simply did not have enough to learn from. Switching to use **all 644 images** was the single change that made structured generation possible at all; everything else was fine-tuning on top of that. Even so, 644 is still small, which is exactly why the final samples are suggestive rather than crisp. If I could change one thing to improve results, it would be more data, not more tuning.

5.4 EMA of the weights

Sampling from an exponential moving average of the weights, instead of the raw training weights, gave cleaner and steadier images. There was a catch in the decay rate, though: a very high decay of **0.9999** updated the averaged weights too slowly for this dataset's modest number of steps, so they stayed partly stuck at their random starting point and produced ugly red-and-black outputs. Turning the decay down to **0.999** fixed it.

5.5 Noise schedule

Swapping the original linear noise schedule for a **cosine** one gave a better signal at the low-noise end of the process and contributed to both the smoother training and the improved structure in the samples.

5.6 Data augmentation

Since data was the bottleneck, I added augmentation, resize-and-random-crop, horizontal flips, and a little colour jitter, to squeeze more variety out of the images on hand. This helped the model generalise instead of just memorising the same few hundred pictures.

6. A Look at the In-Between Stage

The epoch-150 checkpoint sits nicely between the two stages above, the colour fields are starting to firm up into shapes but haven't fully resolved yet. If there are more sample grids to include, the placeholder below can be swapped for them.



Figure 4. Epoch 150, an intermediate stage, structure beginning to emerge.

7. Wrapping Up

Training a diffusion model from scratch on just 644 images of five animals worked better than I expected, the final samples are coherent, natural-looking scenes with recognisable animal-like forms, even if they aren't photo-sharp. The three things I'll remember from it: the loss saturates almost immediately and tells you very little about how good the images are; the amount of data mattered more than any single hyperparameter, with the jump from ~ 100 to 644 images being the difference between noise and real pictures; and a handful of stability choices, a lower learning rate, plain cosine decay, gradient clipping, the right EMA decay, and a cosine noise schedule, were each needed to get clean training. The clearest path to better results from here would simply be more images per class.